

Міністерство освіти і науки України
Чернівецький національний університет
імені Юрія Федьковича

Факультет математики та інформатики
(повна назва інституту/факультету)

Кафедра прикладної математики та інформаційних технологій
(повна назва кафедри)

Поліноміальний алгоритм завадостійкого кодування

Курсова робота

Рівень вищої освіти - перший (бакалаврський)

Виконав:

студент 3 курсу, групи 312

спеціальності 1.113 – Прикладна математика
(назва спеціальності)

Порчук Іван Русланович

(прізвище, ім'я та по-батькові)

Науковий керівник: канд.техн.н.,

доцент Шепетюк Б.Д.

(науковий ступінь, вчене звання, прізвище та ініціали)

Чернівці – 2025

АНОТАЦІЯ

У курсовій роботі створено інтерактивну програму реалізації алгоритмів економного і поліноміального кодування та обчислення синдрому та виправлення одиночних помилок в циклічних кодах.

Зміст

ВСТУП.....	5
1. ТЕОРЕТИЧНА ЧАСТИНА	7
1.1. Завадостійке кодування	7
1.2. Поліноміальні коди	11
1.3 Циклічні коди	13
1.4 Кодування з використанням циклічних кодів	14
1.5 Обчислення синдрому та виправлення помилок в циклічних кодах	16
2. ПРАКТИЧНА ЧАСТИНА	20
2.1 Постановка задачі.....	20
2.2. Виявлення і виправлення помилок в циклічних кодах	21
2.3. Програмна реалізація	26
ВИСНОВКИ.....	32
ПЕРЕЛІК ПОСИЛАНЬ	33
ДОДАТОК А.....	34

ВСТУП

Сьогодні в сучасному житті ми постійно виконуємо певні дії з передачі інформації [1]. Наприклад, коли спілкуємось за допомогою мобільного телефону, замовляємо потрібні нам товари в електронних магазинах, скачуємо фільми або улюблену музику. В усіх цих випадках потрібні нам повідомлення подорожують кабельними або ефірними каналами зв'язку, а також тимчасово або постійно зберігаються в пам'яті хмарних серверів чи власних комп'ютерів і пристроїв персонального корис. При такій подорожі інформації по каналах зв'язку на кожному кроці використовуються різноманітні технічні рішення, що забезпечують необхідну ефективність передачі і зберігання даних.

Теоретичні підходи даного процесу передачі і зберігання інформації базуються на математичних моделях, що відображують вплив параметрів передачі на три основних показники її ефективності — *компактність, швидкість та достовірність*. Ці три критерії є найбільш універсальними, вони властиві будь-яким видам передачі повідомлень в просторі або в часі.

Практичні рішення щодо ефективності передачі інформації в сучасних умовах втілюються здебільшого у вигляді алгоритмів та програм кодування та обробки сигналів. Вони реалізуються зазвичай в формі програмних продуктів та апаратних рішень, що певним чином конкурують між собою та постійно удосконалюються. Однак основою цих рішень є певні математичні методи і підходи, які лишаються незмінними. Саме таким методам присвячена дана робота.

Метою даної курсової роботи є створення зручного програмного забезпечення для моделювання основних властивостей алгоритмів економного і завадостійкого кодування.

Об'єктом дослідження є основні алгоритми економного та завадостійкого кодування інформації. При дослідженні використовується комп'ютерне моделювання на основі існуючих математичних моделей.

Практичне значення курсової роботи полягає у наступному:

розроблений додаток може практично використовуватись на кафедрі прикладної математики та інформаційних технологій ФМІ ЧНУ імені Юрія Федьковича при читанні вибіркового курсу «Теорія інформації та кодування».

1. ТЕОРЕТИЧНА ЧАСТИНА

1.1. Завадостійке кодування

Розглянемо основні поняття, що пов'язані з кодуванням та передаванням інформації. Подією називатимемо кожен кількісний чи якісний визначений стан динамічної системи, який фіксується спостереженнями. Можна кожному стану системи поставити у відповідність певне значення чи послідовність значень деякої величини. За допомогою цієї величини можна здійснити передавання повідомлення (відомостей про подію, інформації про подію) від одного об'єкта до іншого. Фізичний процес, що являє собою матеріальне втілення повідомлення, називається сигналом. Система або середовище, де здійснюється передавання сигналу, називається каналом зв'язку. Будь-які повідомлення, що підлягають передаванню по каналах зв'язку, переробці в кібернетичній системі, мають бути попередньо закодовані, тобто «перекладені» мовою сигналів [3,4].

Кодування можна визначити як процес подання інформації у вигляді деякої послідовності символів (кодових комбінацій). При цьому таку послідовність, у свою чергу, можна подати (перекодувати) у вигляді сукупностей фізичних сигналів тієї чи іншої природи — акустичних, оптичних, електричних тощо.

Наведемо приклад природного кодування. Нехай ви спостерігаєте деякий пейзаж. До вашого ока надходить інформація про це у вигляді світлових сигналів (фотонів). Ці сигнали сітківкою ока перекоднуються в інші сигнали, що по нейронних ланцюгах надходять до головного мозку. Там ці сигнали перекодовуються в образи, які далі перекодовуються в певні відчуття.

Проте якщо потрібно цю інформацію зафіксувати на папері, її доводиться перекодувати у вигляді букв та їх поєднань. А щоб цю інформацію повідомити комусь по телефону, її необхідно ще перекодувати у звукові коливання. Потім телефон ще раз закодує звукові коливання в електричні імпульси, які по телефонних лініях (каналах зв'язку) надійдуть на приймальний пристрій адресата, де відбудеться декодування електричних імпульсів у звукові

коливання. Нарешті, ці коливання надійдуть адресатові (до його вуха), де він їх декодує в образи, або в текст.

Наведемо більш строге визначення кодування. Нехай дано довільну множину A , яку потрібно відобразити в іншу множину B . У цій множині B є скінченна кількість символів (знаків), що називається алфавітом. Наприклад, в абетці Морзе три символи (крапка, тире і прогалина), в англійській мові — 26 букв плюс прогалина і т. ін.

Кількість різних символів (букв), що входять до алфавіту, називається обсягом алфавіту. У цій множині B за певними правилами можна будувати послідовності символів, що називаються словами.

Кодуванням називається відображення довільної множини A у множину скінчених послідовностей (слів), утворених за допомогою деякого алфавіту множини B , а декодуванням — обернене відображення.

Кодом називається сукупність знаків (символів) алфавіту B і слів, складених із них за певними правилами і призначених для однозначного відображення множини A у множину B .

До будь-якої системи кодування висуваються такі основні вимоги:

1) взаємна однозначність перетворень відображуваної множини A у множину B , що її відображує в результаті кодування та оберненого перетворення (декодування) — необхідна умова відсутності помилок в інтерпретації вихідної інформації;

2) економічність кодування, забезпечується, насамперед, мінімізацією середньої довжини комбінацій, а отже, і довжини інформаційних текстів, завдяки чому заощаджується не лише час передавання тексту, а і носії інформації;

3) завадостійкість, тобто можливість виявлення та виправлення помилок у кодових комбінаціях під впливом тих чи інших перешкод та збоїв.

Зауважимо, що друга і третя вимоги взаємно суперечливі, оскільки підвищення збоєстійкості кодів досягається збільшенням довжини слів, через що знижується економічність систем кодування.

У техніці зв'язку й обробки інформації розроблено багато різних способів кодування, що забезпечують більш-менш вдалий компроміс у виконанні цих вимог у різних кібернетичних системах.

Схематичне зображення системи зв'язку наведено на рис. 1. Ця схема відбиває найбільш істотні елементи будь-якої системи зв'язку: комп'ютерної мережі, системи супутникового чи мобільного зв'язку, розмовного каналу між двома співрозмовниками тощо.

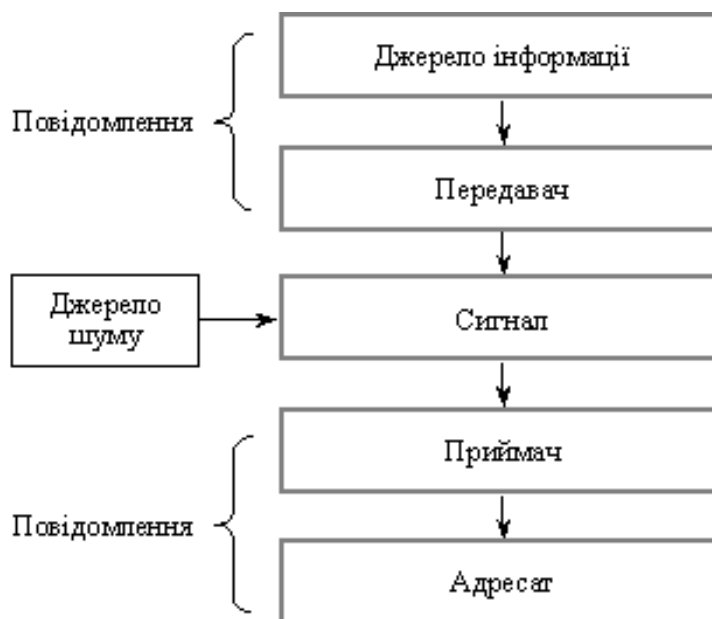


Рис. 1.1. Принципова схема системи зв'язку

Людське мислення у процесі переробки інформації являє собою своєрідний канал зв'язку із шумами, від пропускну здатності якого багато в чому залежить дієвість та ефективність управлінських рішень.

Тоді, коли людина встигає вчасно переробити необхідну для ухвалення рішення інформацію, тобто її канал зв'язку має достатню пропускну здатність і стійкість до шумів, можна очікувати прийняття найбільш ефективних рішень. Якщо терміни переробки та проходження інформації через канал зв'язку (свідомість людини) та час, необхідний для прийняття рішення, не збігаються, то рішення приймається або із запізненням, або в умовах неповної переробки інформації. І перше і друге негативно позначається на функціонуванні соціально-економічних систем.

Поняття про цифровий код. Розглянемо питання про вибір цифрового алфавіту для кодування величин, тобто про вибір системи числення.

У будь-якій позиційній системі числення деяке число N визначається виразом

$$N_{(a)} = \sum_{i=1}^n k_i a^{i-1},$$

де a — основа системи числення (обсяг цифрового алфавіту); n — кількість розрядів у числі, i — порядковий номер розряду; k_i — коефіцієнт, що може набувати різних значень $k_i = 0, 1, 2, \dots, (a - 1)$. При цьому запис числа N у системі з основою a здійснюється у вигляді

$$N_i = k_n, k_{n-1}, k_{n-2}, \dots, k_2, k_1.$$

Наприклад:

$$27(10) = 24 + 23 + 21 + 20 \Rightarrow 27(2) = 11011 = k_5 2^5 - 1 + k_4 2^4 - 1 + k_3 2^3 - 1 + k_2 2^2 - 1 + k_1 2^1 - 1.$$

Зі зменшенням основи a , тобто спрощенням алфавіту відбувається зміна його кодової комбінації. Коли йдеться про передавання та електронну обробку інформації, найзручнішою є двійкова система числення, в якій $a = 2$ і k_i може набувати лише двох значень: 0 і 1. Обчислювальні машини, як і інші технічні пристрої, призначені для збереження і переробки інформації, закодованої в двійковій системі числення.

Кодування є перетворенням повідомлення в послідовність деяких символів. Для цього встановлюють взаємно-однозначну відповідність між повідомленнями і символами, яке називається кодом. Код має бути однаковим як на передавальній, так і на приймальній сторонах системи.

По завадостійкості коди діляться на прості (примітивні) і корегуючі. Коди, в яких всі можливі кодові комбінації використовуються для передачі інформації, називаються простими, або кодами без надмірності. У простих рівномірних кодах перетворення одного символу комбінації в іншій, наприклад 1 в 0 або 0 в 1, наводить до появи нової комбінації, тобто до помилки. Коди, що коректують, будуються так, що для передачі повідомлення використовуються

не все кодові комбінації, а лише деяка частина їх. Тих самим створюється можливість виявлення і виправлення помилки при неправильному відтворенні деякого числа символів. Властивості кодів, що коректують, досягаються ціною введення в кодові комбінації додаткових (надлишкових) символів.

1.2. Поліноміальні коди

Представлення кодового слова (n,k) -коду у вигляді послідовності $U = (U_0, U_1, \dots, U_{n-1})$ довжиною n символів або їх завдання за допомогою системи перевірочних рівнянь і породжуючої матриці не є єдино можливим. Ще один зручний і широко використовуваний способом подання того ж кодового слова полягає в тому, що елементи $U_0, U_1, \dots, U_{n-1}$ є коефіцієнтами многочлена від X , тобто

$$U(x) = f(x) = U_0 + U_1 \cdot X + U_2 \cdot X^2 + \dots + U_{n-1} \cdot X^{n-1} \quad (1.2.1)$$

Використовуючи це подання, можна визначити поліноміальний код як безліч всіх многочленів ступеня, не більше $n-1$, що містять в якості загального множника деякий фіксований многочлен $g(x)$. Многочлен $g(x)$ називається породжуючим многочленом коду. Представлення кодових слів у такій формі дозволяє звести дії над комбінаціями символів до дії над поліномами. Визначимо дії над поліномами в полі двійкових символів $GF(2)$.

Сумою двох поліномів $f(x)$ і $g(x)$ з $GF(2)$ називається поліном з $GF(2)$, що визначається таким чином:

$$f(x) + g(x) = \sum_{i=0}^{n-1} (f_i + g_i) \cdot x^i \quad (1.2.2)$$

Іншими словами, складанню двійкових поліномів відповідає складання по $\text{mod } 2$ коефіцієнтів при однакових степенях x . Наприклад:

$$\begin{array}{r} X^3 + X^2 + 0 \cdot X + 1 \\ X + 1 \\ \hline X^3 + X^2 + X + 0 = X^3 + X^2 + X \end{array}$$

$$\begin{array}{r} X^3 + X^2 + 0 \cdot X + 1 \\ X^2 + X + 1 \\ \hline X^3 + 0 + X + 0 \end{array} = X^3 + X. \quad (1.2.3)$$

Добутком двох поліномів з $GF(2)$ називається поліном з $GF(2)$, що визначається таким чином:

$$f(x) \cdot g(x) = \sum_{i=0}^{n-1} \left(\sum_{j=0}^i f_j \cdot g_{i-j} \right) \cdot x^i, \quad (1.2.4)$$

тобто добуток виходить за звичайним правилом множення степеневих функцій, проте одержувані коефіцієнти при даному степені X складаються за модулем 2.

$$\begin{array}{r} X^3 + X^2 + 0 + 1 \\ X + 1 \\ \hline X^3 + X^2 + 0 + 1 \\ X^4 + X^3 + 0 + X \\ \hline X^4 + 0 + X^2 + X + 1 = X^4 + X^2 + X + 1 \\ X^3 + X^2 + 0 + 1 \\ X^2 + X \\ \hline X^4 + X^3 + 0 + X \\ X^5 + X^4 + 0 + X^2 \\ \hline X^5 + 0 + X^3 + X^2 + X = X^5 + X^3 + X^2 + X. \end{array} \quad (1.2.5)$$

Нарешті, можна сформулювати теорему про поділ поліномів: для кожної пари поліномів $C(x)$ і $d(x)$, $d(x) \neq 0$ існує єдина пара поліномів $q(x)$ - частка і $p(x)$ залишок, такі, що

$$C(x) = q(x) \cdot d(x) + p(x), \quad (1.2.6)$$

де ступінь залишку $p(x)$ менше ступеня дільника $d(x)$.

Іншими словами, поділ поліномів проводиться за правилами поділу степеневих

функцій, при цьому операція віднімання замінюється сумовуванням по $\text{mod}2$.

Наприклад:

$$\begin{array}{r}
 X^4 + 0 + X^2 + X + 1 \quad \Big| \quad X + 1 \\
 \underline{X^4 + X^3} \\
 X^3 + X^2 + X + 1 \\
 \underline{X^3 + X^2} \\
 X + 1 \\
 \underline{X + 1} \\
 0
 \end{array}$$

Ще раз нагадаємо, що при складанні по $\text{mod}2$ сума двох одиниць (тобто двох елементів полінома з однаковими степенями) буде дорівнювати нулю, а не звичним в десятковій системі числення двом. І, крім цього, операції віднімання і додавання по $\text{mod}2$ збігаються.

1.3 Циклічні коди

Практичним і найбільш широко поширеним класом поліноміальних кодів є циклічні коди. Лінійний (n, k) -код називається циклічним, якщо в результаті циклічного зсуву кодового слова виходить інше кодове слово даного коду. Іншими словами, якщо $U = (U_0, U_1, \dots, U_{n-1})$ є кодовим словом, то й $V = (U_{n-1}, U_0, U_1, \dots, U_{n-2})$, отримане циклічним зсувом U , є кодовим словом даного коду. Циклічні коди привабливі з двох причин.

По-перше, операції кодування та обчислення синдрому для них виконуються дуже просто за використанням зсувних регістрів.

По-друге, цим кодам властива алгебраїчна структура, і можна знайти більш прості та ефективні способи їх декодування.

$$g(x) = 1 + g_1 \cdot x + g_2 \cdot x^2 + \dots + g_{n-k-1} \cdot x^{n-k-1} + x^{n-k}, \quad (1.3.1)$$

Основні властивості циклічних кодів:

1. У циклічному (n, k) -коді кожен ненульовий поліном повинен мати степінь, принаймні $(n-k)$, але не більше $n-1$.

2. Існує тільки один кодовий поліном $g(x)$ степеня $(n-k)$ виду так званий породжуючий поліномом коду.

3. Кожен кодовий поліном $U(x)$ є кратним $g(x)$, тобто

$$U(x) = m(x) \cdot g(x). \quad (1.3.2)$$

1.4 Кодування з використанням циклічних кодів

Припустимо, треба закодувати деяку інформаційну послідовність

$$m = (m_0, m_1, m_2 \dots m_{k-1}). \quad (1.4.1)$$

Відповідний їй поліном виглядає наступним чином:

$$m(x) = m_0 + m_1 \cdot x + m_2 \cdot x^2 + \dots + m_{k-1} \cdot x^{k-1}. \quad (1.4.2)$$

Помноживши $m(x)$ на x^{n-k} :

$$x^{n-k} \cdot m(x) = m_0 \cdot x^{n-k} + m_1 \cdot x^{n-k+1} + \dots + m_{k-1} \cdot x^{n-1}, \quad (1.4.3)$$

отримаємо поліном степеня $n-1$ або менше.

Скориставшись теоремою про ділення поліномів, можна записати

$$x^{n-k} \cdot m(x) = q(x) \cdot g(x) + \rho(x), \quad (1.4.4)$$

де $q(x)$ і $\rho(x)$ - частка і залишок від ділення полінома $x^{n-k} \cdot m(x)$ на породжуючий поліном $g(x)$. Оскільки степінь $g(x)$ дорівнює $(n-k)$, то степінь $\rho(x)$ повинна бути $(n-k-1)$ або менше, а сам поліном $\rho(x)$ буде мати вигляд

$$\rho(x) = \rho_0 + \rho_1 \cdot x + \rho_2 \cdot x^2 + \dots + \rho_{n-k-1} \cdot x^{n-k-1}. \quad (1.4.5)$$

З урахуванням правил арифметики в $GF(2)$ дане вираження можна переписати наступним чином:

$$\rho(x) + x^{n-k} \cdot m(x) = q(x) \cdot g(x), \quad (1.4.6)$$

звідки видно, що поліном $\rho(x) + x^{n-k} \cdot m(x)$ є кратним $g(x)$ і має степінь $n-1$ або меншу. Отже, поліном $\rho(x) + x^{n-k} \cdot m(x)$ - це кодовий поліном, відповідної

кодованої інформаційної послідовності $m(x)$. Розкривши останній вираз, отримаємо

$$\rho(x) + m(x) \cdot x^{n-k} = \rho_0 + \rho_1 x + \rho_2 x^2 + \dots + \rho_{n-k-1} x^{n-k-1} + m_0 x^{n-k} + m_1 x^{n-k+1} + \dots + m_{k-1} x^{n-1},$$

що відповідає кодовому слову

$U =$	$(\rho_0, \rho_1 \dots \rho_{n-k-1},$	$m_0, m_1 \dots m_{k-1}),$
	<i>перевірочні символи</i>	<i>інформаційні символи</i>

Таким чином, кодове слово складається з незмінною інформаційної частини m , перед якою розташовано $(n-k)$ перевірочних символів. Перевірочні символи є коефіцієнтами полінома $\rho(x)$, тобто залишку від ділення $m(x) \cdot x^{n-k}$ на породжуючий поліном $g(x)$. Щоб отриманий результат був зрозуміліше, нагадаємо, що множенню деякого двійкового полінома на x^{n-k} відповідає зсув двійкової послідовності $m = (m_0, m_1 \dots m_{k-1})$ на $n-k$ розрядів вправо. Розглянемо приклад. З використанням коду, що задається породжуючим поліномом $g(x) = 1 + x + x^3$, закодуємо довільну послідовність, наприклад $m = (0111)$. Послідовності $m = (0111)$ відповідає поліном $m(x) = x + x^2 + x^3$. Помножимо $m(x)$ на x^{n-k} :

$$m(x) \cdot x^{n-k} = m(x) \cdot x^3 = (x + x^2 + x^3) \cdot x^3 = x^4 + x^5 + x^6. \quad (1.4.7)$$

Розділимо $m(x) \cdot x^{n-k}$ на породжуючий поліном $g(x)$:

$$\begin{array}{r|l} X^6 + X^5 + X^4 & X^3 + X + 1 \\ \hline X^6 + 0 + X^4 + X^3 & X^3 + X^2 = g(x) \\ \hline X^5 + 0 + X^3 & \\ \hline X^5 + 0 + X^3 + X^2 & \\ \hline X^2 = \rho(x). & \end{array} \quad (1.4.8)$$

Таким чином, кодовий поліном, відповідної інформаційної послідовності $m = (0111)$, буде мати наступний вигляд:

$$U(x) = 0 \cdot X^0 + 0 \cdot X^1 + 1 \cdot X^2 + 0 \cdot X^3 + 1 \cdot X^4 + 1 \cdot X^5 + 1 \cdot X^6, \quad (1.4.9)$$

а відповідне кодове слово $U = (0010111)$.

Отже, циклічний (n, k) -код k -розрядної інформаційної послідовності

$m = (m_0, m_1 \dots m_{k-1})$ одержують у такий спосіб:

- Інформаційну послідовність m множать на x^{n-k} , тобто зсувають вправо на $n-k$ розрядів;

Поліном отриманої послідовності ділять на породжуючий поліном коду $g(x)$;

Отриманий залишок від ділення $m(x) \cdot x^{n-k}$ на $g(x)$, додають до $m(x) \cdot x^{n-k}$, тобто записують в молодших $n-k$ розрядах коду.

Ще одною важливою властивістю циклічного (n, k) -коду, що випливає з теореми про існування циклічних кодів, є те, що його породжуючий поліном ділить без залишку двочлен $x^n + 1$, тобто

$$x^n + 1 = g(x) \cdot h(x) + 0. \quad (1.4.10)$$

Поліном $h(x)$ - частка від ділення $x^n + 1$ на $g(x)$ - називається перевірочним поліномом. Оскільки $h(x)$ однозначно пов'язаний з $g(x)$, він також визначає код. Отже, за допомогою перевірочного полінома $h(x)$ теж можна робити кодування

Виробляється $(n-k)$ зсувів, при цьому формуються і видаються в канал $(n-k)$ перевірочних символів. Для циклічного $(7,4)$ -коду з породжуючим поліномом $g(x) = 1 + x + x^3$ перевірочний поліном $h(x)$ має вигляд

$$h(x) = \frac{x^{n-1}}{1 + x + x^3} = 1 + x + x^2 + x^4. \quad (1.4.11)$$

1.5 Обчислення синдрому та виправлення помилок в циклічних кодах

Обчислення синдрому для циклічних кодів є досить простою процедурою. Нехай $U(x)$ і $r(x)$ поліноми, що відповідають передуваному кодовому слову та прийнятій послідовності. Поділивши $r(x)$ на $g(x)$, отримаємо

$$r(x) = q(x) \cdot g(x) + s(x), \quad (1.5.1)$$

де - $q(x)$ - частка від ділення, $s(x)$ - залишок від ділення.

Якщо $r(x)$ є кодовим поліномом, то він ділиться на $g(x)$ без залишку, тобто $s(x) = 0$. Отже, $s(x) \neq 0$ є умовою наявності помилки у прийнятій

послідовності, тобто синдромом прийнятої послідовності. Синдром $s(x)$ має в загальному випадку вид

$$S(x) = S_0 + S_1 \cdot x + \dots + S_{n-k-1} \cdot x^{n-k-1} \quad (1.5.2)$$

Очевидно, що схема обчислення синдрому є схемою ділення, подібною схемам кодування рисунках 1.4.1 та 1.4.2. При наявності у прийнятій послідовності r хоча б однієї помилки вектор синдрому S буде мати, принаймні, один нульовий елемент, при цьому факт наявності помилки легко виявити, об'єднавши з **АБО** виходи всіх комірок регістра синдрому.

Покажемо, що синдромний многочлен $S(x)$ однозначно пов'язаний з многочленом помилки $e(x)$, а значить, з його допомогою можна не тільки виявляти, але й локалізувати помилку в прийнятій послідовності. Нехай

$$e(x) = e_0 + e_1 \cdot x + e_2 \cdot x^2 + \dots + e_{n-1} \cdot x^{n-1} \quad (1.5.3)$$

- поліном вектора помилки. Тоді поліном прийнятої послідовності

$$r(x) = U(x) + e(x). \quad (5.4)$$

Додамо в цьому вираженні ліворуч і праворуч $U(x)$, а також врахуємо, що

$$r(x) = q(x) \cdot g(x) + S(x), \quad U(x) = m(x) \cdot g(x), \quad (1.5.5)$$

тоді

$$e(x) = [m(x) + q(x)] \cdot g(x) + S(x) = f(x) \cdot g(x) + S(x), \quad (1.5.6)$$

Тобто, синдромний поліном $S(x)$ є залишком від ділення полінома помилки $e(x)$ на породжуючий поліном $g(x)$. Звідси випливає, що по синдрому $S(x)$ можна однозначно визначити вектор помилки $e(x)$, а отже, виправити цю помилку.

Неалгебраїчні методи декодування циклічних кодів

Всі методи декодування лінійних блочних кодів можна розбити на дві групи: алгебраїчні і неалгебраїчні. В основі алгебраїчних методів лежить

розв'язання систем рівнянь, які задають значення і розташування помилок. Розглянуті синдромні декодери відносяться саме до цієї групи методів. При неалгебраїчних методах та ж мета досягається за допомогою простих структурних понять теорії кодування, що дозволяють знаходити комбінації помилок більш простим шляхом. Одним з неалгебраїчних методів є декодування з використанням алгоритму Меггітта, придатного для виправлення як поодиноких, так і l -кратних помилок (на практиці $l \leq 3$)

При декодуванні відповідно до алгоритму Меггітта також обчислюється синдром прийнятої послідовності $S(x)$, однак використовується він інакше, ніж у розглянутих раніше синдромних декодерах. Ідея, що лежить в основі декодера Меггітта, дуже проста і ґрунтується на наступних властивостях циклічних кодів:

Існує взаємно-однозначна відповідність між множиною всіх виправлених помилок і множиною синдромів;

- Якщо $S(x)$ – синдромний многочлен, відповідаючий многочлену помилок $e(x)$, то $xS(x) \bmod g(x)$ – синдромний многочлен, відповідаючий

$x \cdot e(x) \bmod (x^n + 1)$. Рівність $a(x) = b(x) \bmod p(x)$ читається як " $a(x)$, можна порівняти з $b(x)$ за модулем $p(x)$ " і означає, що $a(x)$ і $b(x)$ мають однакові залишки від ділення на поліном $p(x)$.

Поліном помилки в старшому розряді для (7,4)-коду Хеммінга має вигляд $e_6(x) = x^6$, відповідний йому синдромний поліном $S_6(x) = 1 + x^3$ ($S = (101)$), детектор помилки налаштований на це значення синдрому. Нехай, наприклад, передана послідовність $U = (1001011)$, їй відповідає кодовий поліном $U(x) = 1 + x^3 + x^5 + x^6$. Сталася помилка в третій позиції $e(x) = x^3$, прийнятий вектор має вигляд $r = (1000011)$, його поліном $r(x) = 1 + x^5 + x^6$. Якщо прийнята послідовність повністю входить в буферний регістр, в регістрі синдрому виявляється синдром, що відповідає помилці $e(x) = x^3 S_3 = (110)$. Оскільки він не збігається з табличним для e_6 , на виході детектора помилок буде 0 і виправлення не відбувається. Далі виробляються однократний циклічний зсув прийнятої послідовності в буферному регістрі, одноразовий

поділ синдрому $x \cdot S_3$ на породжуючий поліном в регістрі зі зворотними зв'язками і перевірка на збіг синдрому із заданим. Послідовність станів регістрів декодера в процесі декодування показана на рис.1.5.1.

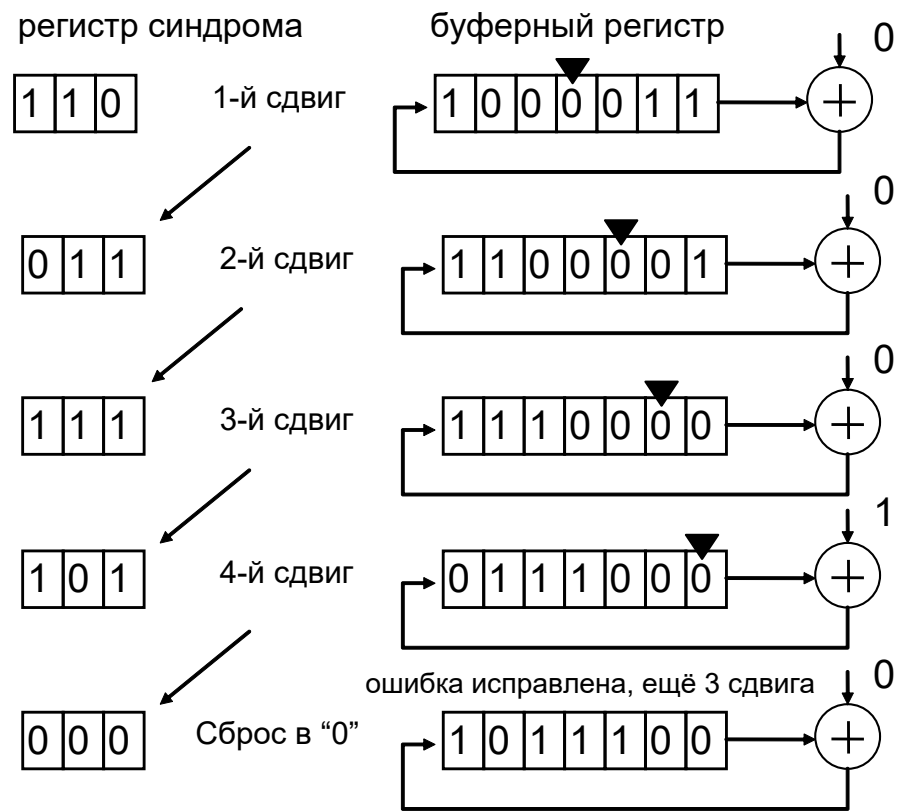


Рис. 1.5.1. Декодер Меггітта.

Таким чином, виправлення помилки в декодері Меггітта здійснюється за $2 \cdot n$ тактів: протягом n -тактів проводиться введення прийнятої послідовності в буферний регістр, протягом l тактів - виправлення помилки, і ще протягом $n - l$ - відновлення буферного регістра в початковий стан з виправленим словом. Простота декодера досягається збільшенням часу декодування. Слід зазначити, що у зв'язку з успіхами в розробці БІС і пристроїв пам'яті в значній мірі знімається питання про розміри таблиць, що зв'язують значення синдрому і вектора помилки (для синдромних декодерів) та навіть значення кодових слів і прийнятих послідовностей (для декодера максимального правдоподібності). Тому в перспективі можливе зниження інтересу до кодів, що володіють спеціальною структурою, і до методів їх декодування.

2. ПРАКТИЧНА ЧАСТИНА

2.1 Постановка задачі

В даній курсовій роботі засобами Delphi було розроблено програмний продукт, який кодує довільну інформацію за допомогою двох алгоритмів (Шенона-Фано і Хафмана), а також запрограмовано алгоритм виправлення одиночної помилки у циклічних кодах Хемінга (7,4). Дана програма була розроблена в Borland Delphi.

Вага і відстань Хемінга. Здатність коду виявляти і виправляти помилки

Розглянемо, чим визначається здатність блокової коду виявляти і виправляти помилки, що виникли при передачі.

Хай $U = (U_0, U_1, U_2, \dots, U_{n-1})$ - двійкова послідовність довжиною n .

Число одиниць (ненульових компонент) в цій послідовності називається вагою Хемінга вектора U і позначається $w(U)$.

Наприклад, вага Хемінга вектора $U = (1001011)$ рівний чотирьом, для вектора $U = (1111111)$ величина $w(U)$ складе 7 і так далі

Таким чином, ніж більше одиниць в двійковій послідовності, тим більше її вага Хемінга.

Далі, хай U і V будуть двійковими послідовностями довжиною n .

Число розрядів, в яких ці послідовності розрізняються, називається відстанню Хемінга між U і V і позначається $d(U, V)$.

Наприклад, якщо $U = (1001011)$, а $V = (0100011)$, то $d(U, V) = 3$.

Задавши лінійний код, тобто визначивши все 2^k його кодових слів, можна обчислити відстань між всіма можливими парами кодових слів.

Мінімальне з них називається мінімальною кодовою відстанню коди і позначається d_{min} .

Можна перевірити і переконатися, що мінімальна кодова відстань для того, що розглядається нами в прикладах (7,4) - коду рівно трьом: $d_{min(7,4)} = 3$. Для цього потрібно записати всі кодові слова (7,4) -кода Хемінга (всього 16 слів), обчислити відстані між їх всіма парами і узяти найменше значення. Проте можна визначити d_{min} блокової коди і простішим способом.

Доведено, що відстань між нульовим кодовим словом і одним з кодових слів, що входять в матрицю (рядки матриці лінійної блокової коди, що породжує, самі є кодовими словами, за визначенням), що породжує, рівно d_{min} . Але відстань від будь-якого кодового слова до нульового дорівнює вазі Хемінга цього слова. Тоді d_{min} рівно мінімальній вазі Хемінга для всіх рядків матриці коду, що породжує.

Якщо при передачі кодового слова по каналу зв'язку в нім сталася одиночна помилка, то відстань Хемінга між переданим словом U і прийнятим вектором r дорівнюватиме одиниці. Якщо при цьому одне кодове слово не перейшло в інше (а при $d_{min} > 1$ і при одиночній помилці це неможливо), то помилка буде виявлена при декодуванні.

У загальному випадку якщо блоковий код має мінімальну відстань d_{min} , то він може виявляти будь-які поєднання помилок при їх числі, меншому або рівному $d_{min} - 1$, оскільки жодне поєднання помилок при їх числі, меншому, ніж $d_{min} - 1$, не може перевести одно кодове слово в друге.

Але помилки можуть мати кратність і більшу, ніж $d_{min} - 1$, і тоді вони залишаються невиявленими.

2.2. Виявлення і виправлення помилок в циклічних кодах

Помилки в циклічних кодах виявляються і виправляються за допомогою залишків від ділення отриманої комбінації на породжуючий многочлен. Залишки від ділення є пізнавачами помилок, але не вказують безпосередньо на місце помилки в циклічному коді.

Ідея виправлення помилок базується на тому, що помилкова комбінація після певного числа циклічних зсувів «підганяється» під залишок таким чином, що в сумі із залишком вона дає виправлену комбінацію. Залишок при цьому є не що інше, як різниця між спотвореними і правильними символами, одиниці в залишку стоять на місцях спотворених розрядів в «підігнаній» циклічними зсувами комбінації. Підганяють же спотворену комбінацію до тих пір, поки число одиниць в залишку не дорівнюватиме числу помилок, яке ще здатне виправити даний код. При цьому, природно, число одиниць може дорівнювати числу помилок S , що виправляються даним кодом (код виправляє три помилки і в спотвореній комбінації три помилки), або менше S (код виправляє три помилки, а в прийнятій комбінації - одна помилка).

Місце помилки в кодовій комбінації не має значення. Досить отримати один залишок, вагу якого $W > S$, і цього вистачає для виправлення спотвореної комбінації. У цьому сенсі циклічні коди можуть виправляти пачки помилок, аби довжина пачки не перевищувала S .

Процедура виправлення помилок розглядається на прикладі побудови кодів, що виправляють одиночну помилку, $d_0=3$.

Декодування конкретних циклічних кодів зводиться до наступних стандартних процедур.

1. Розрахунок співвідношення між контрольними і інформаційними символами коду.

Якщо задано число інформаційних розрядів n_i , то число контрольних розрядів n_k знаходиться з виразу

$$n_k = \lceil \log_2 \{ (n_i + 1) + \lceil \log_2 (n_i + 1) \rceil \} \rceil, \quad (1)$$

загальне число символів коду

$$n = n_i + n_k.$$

Якщо задана довжина коду n , то число контрольних розрядів

$$n_k = \lceil \log_2 (n + 1) \rceil. \quad (2)$$

Співвідношення числа контрольних і інформаційних символів для код з $d_0=3$ приведені в таблиці 2 додатку.

2. Вибір породжуючого многочлена здійснюється по таблицях незвідних двійкових многочленів (табл. 2.1).

Породжуючий многочлен $K(X)$ слід вибрати як можна коротшим, але, з одного боку, степінь його має бути не менше числа контрольних розрядів, з іншої - число ненульових членів $K(X)$ має бути не менше мінімальної кодової відстані d_0 .

Коди, отримані при використанні незвідних многочленів вигляду X^3+X^2+1 і X^3+X+1 , що подібні до один одного і володіють рівноцінними здібностями, що коректують. Самі ж многочлени 1101 і 1011 називають зворотними, або двоїстими многочленами. Якщо даний многочлен незвідний, то незвідний буде і двоїстий йому многочлен.

3. Виявлення і виправлення помилок.

Виявлення і виправлення помилок відбувається по залишках від ділення прийнятої комбінації $F(X)$ на породжуючий многочлен $K(X)$. Якщо прийнята комбінація ділиться на породжуючий многочлен без залишку, то код прийнятий безпомилково. Залишок від ділення свідчить про помилку, але не визначає, яка саме. Аби знайти помилковий розряд і виправити його в циклічних кодах, прийнято здійснювати наступні процедури:

а) прийнята комбінація ділиться на породжуючий многочлен;

б) підраховується кількість одиниць в залишку (вага залишку). Якщо $W > S$, де S – допустиме число що виправляються даним кодом помилок, то прийнята комбінація сумується по модулю 2 з отриманим залишком. Сума дає виправлену комбінацію. Якщо $W > S$, то

в) ділимо отриману в результаті циклічного зсуву комбінацію на породжуючий многочлен $K(X)$. Якщо в залишку $W > S$, то складаємо ділене з остачою. Потім проводимо циклічний зсув вправо комбінації, одержаної в результаті підсумовування останнього діленого із залишком. Одержана комбінація вже не містить помилок. Якщо після першого циклічного зсуву і подальшого ділення залишок виходить таким, що його вага $W > S$, то

г) повторюється процедура в) до тих пір, поки не буде $W \leq S$. У цьому випадку комбінація, отримана в результаті останнього циклічного зсуву, сумується із залишком від ділення цієї комбінації на утворюючий многочлен, а потім

д) проводиться циклічний зсув вправо на стільки розрядів, на скільки була зсунута підсумовувана з останнім залишком комбінація відносно прийнятої. В результаті отримаємо виправлену комбінацію.

Приклад. Процес виправлення одиночної помилки в прийнятій кодовій комбінації.

Рішення: Допустимо, передавалася комбінація (1001110) і в ній спотворився 4 розряд. Таким чином, прийнята комбінація має вигляд 1000110.

1. Ділимо прийняту комбінацію на породжуючий многочлен (X^3+X+1)

$$\begin{array}{r}
 1000110 \, | \, \underline{1011} \\
 \underline{1011} \\
 1111 \\
 \underline{1011} \\
 1000 \\
 \underline{1011} \\
 11
 \end{array}$$

3. Порівнюємо вагу отриманого залишку W з можливим для даного коду числом помилок S , що виправляються. Вага залишку $W=2$. Число помилок $S=1$, що виправляються, оскільки $W > S$.

4. Проводимо циклічний зсув прийнятої комбінації $F(X)$ на один розряд вліво з подальшим діленням отриманої в результаті циклічного зсуву комбінації на $K(X)$:

$$\begin{array}{r}
 1000110 \quad 0001101 \, | \, \underline{1011} \\
 \underline{1011} \\
 110 \quad W > S, \text{ тобто } W=2, W > S.
 \end{array}$$

5. Повторюємо процедуру п. 3 до тих пір, поки не буде $W \leq S$

0011010 | 1011

1011

1100

1011

111

W>S;

0110100 | 1011

1011

1100

1011

1110

1011

101

W>S;

1101000 | 1011

1011

1100

1011

1110

1010

1010

1011

1 W = S = 1.

6. Складаємо по модулю 2 останнє ділене з останнім залишком:

1101000

+ 1

1101001

7. Проводимо циклічний зсув комбінації, отриманої в результаті сумування останнього діленого з останнім залишком, **вправо на 4 розряди** (оскільки перед цим ми **чотири** рази зсували прийняту комбінацію **вліво**)

1101001 1110100, 0111010, 0011101, 1001110,

як видно, остання комбінація відповідає переданій, тобто вже не містить помилок.

Табл.2.1. Незвідні поліноми

g(x)	поліном	g(x)	поліном
g(x)	x+1	g ₁ (x ⁶)	x ⁶ +x+1
g(x ²)	x ² +x+1	g ₂ (x ⁶)	x ⁶ +x ³ +1
g ₁ (x ³)	x ³ +x ¹ +1	g ₃ (x ⁶)	x ⁶ +x ⁵ +1
g ₂ (x ³)	x ³ +x ² +1		
g ₁ (x ⁴)	x ⁴ +x+1	g ₁ (x ⁷)	x ⁷ +x+1
g ₂ (x ⁴)	x ⁴ +x ³ +1	g ₂ (x ⁷)	x ⁷ +x ³ +1
g ₃ (x ⁴)	x ⁴ +x ³ +x ² +x+1	g ₃ (x ⁷)	x ⁷ +x ³ +x ² +x+1
g ₁ (x ⁵)	x ⁵ +x ² +1		
g ₂ (x ⁵)	x ⁵ +x ³ +1	g ₁ (x ⁸)	x ⁸ +x ⁴ +x ³ +x+1
g ₃ (x ⁵)	x ⁵ +x ³ +x ⁴ +x+1	g ₂ (x ⁸)	x ⁸ +x ⁴ +x ³ +x ² +1
g ₄ (x ⁵)	x ⁵ +x ⁴ +x ² +x+1		
g ₅ (x ⁵)	x ⁵ +x ⁴ +x ³ +x+1	g ₁ (x ⁹)	x ⁹ +x+1
g ₆ (x ⁵)	x ⁵ +x ⁴ +x ³ +x ² +1	g ₂ (x ⁹)	x ⁹ +x ⁴ +1

Породжуючий поліном вибирають таким чином. По заданій кодовій комбінації k визначають число контрольних символів із співвідношення $r = \log(n + 1)$ або по емпіричній формулі

$$r = [\log\{(k+1) + [\log(k+1)]\}]$$

Співвідношення значень n , k , r можна визначити по табл. 2.2.

Табл. 2. 2. Залежність між n , k , r .

n	3	5	6	7	9...15	17...31	33...63	65...127
k	1	2	3	4	5...11	12...26	27...57	28...120
r	2	3	3	3	4	5	6	7

З таблиці незвідних поліномів (табл.1) вибирають найкоротший многочлен з степенем, рівним числу контрольних символів; його і приймають за породжуючий поліном.

2.3. Програмна реалізація

Стартове вікно (вибирається опція – економне кодування) виглядає наступним чином:

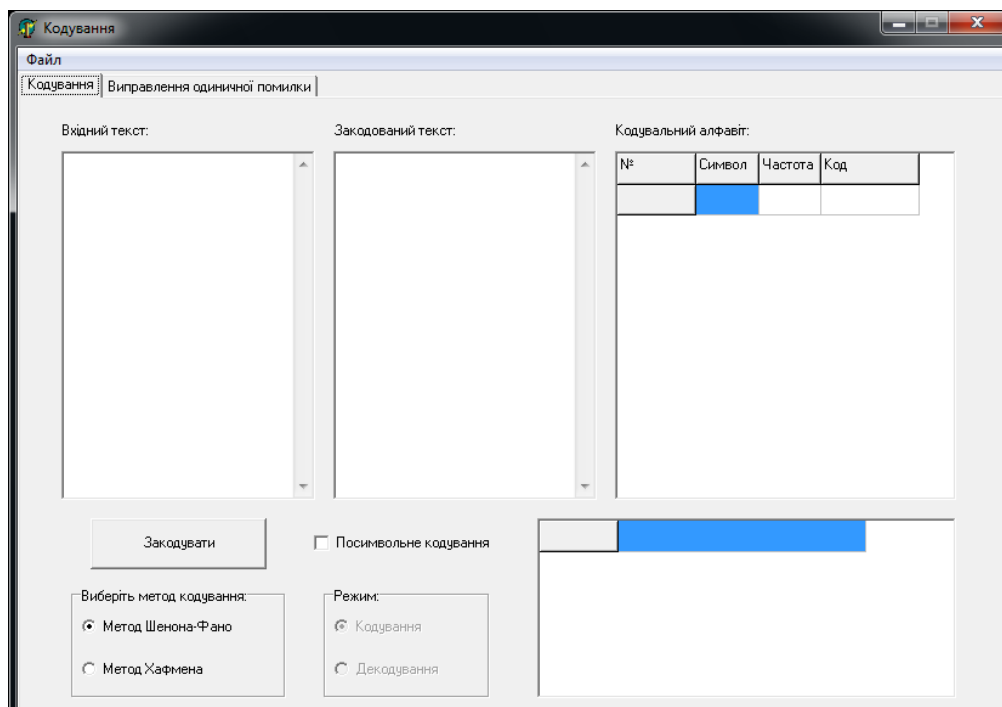


Рис.2.1 Початкове вікно програми

Тут знаходяться 2 вкладки. Перша для економного кодування і декодування інформації. Друга вкладка - для кодування циклічного коду і виправлення одиночної помилки при передачі даного коду по інформаційному каналі.

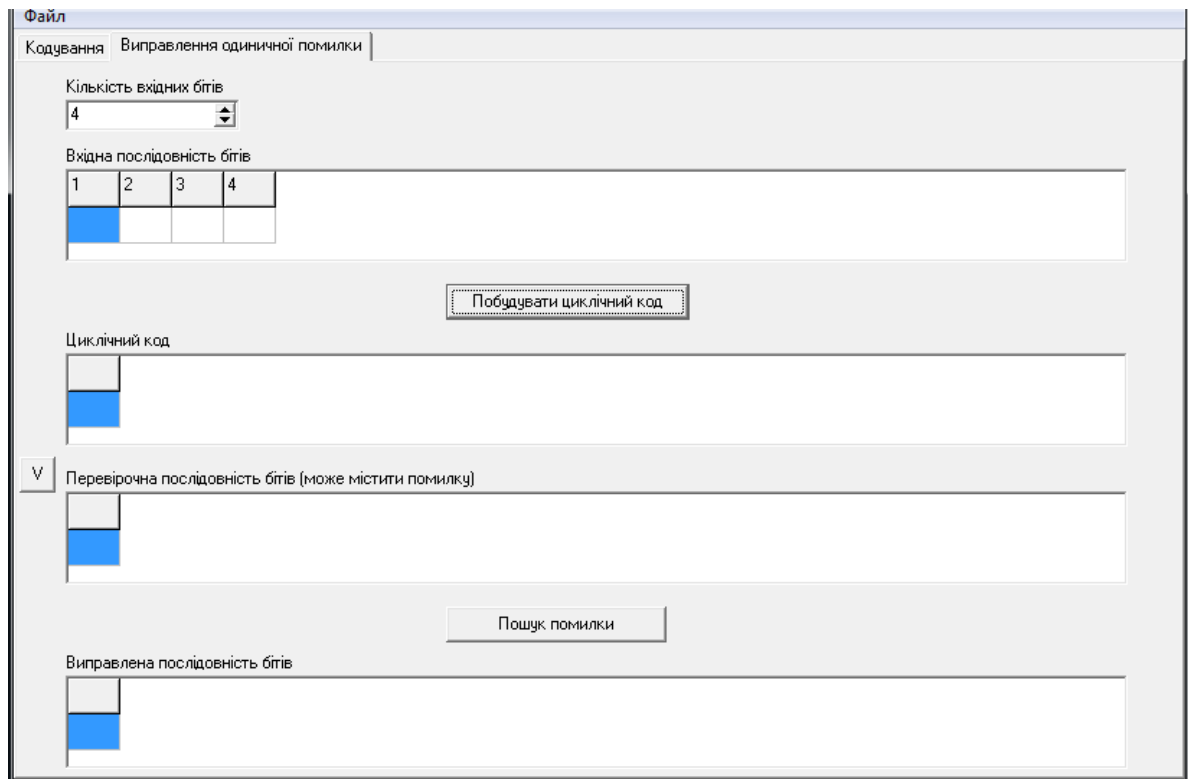


Рис.2.2. Вікно програми для роботи з циклічними кодами
Розглянемо приклад роботи програми (економне кодування).

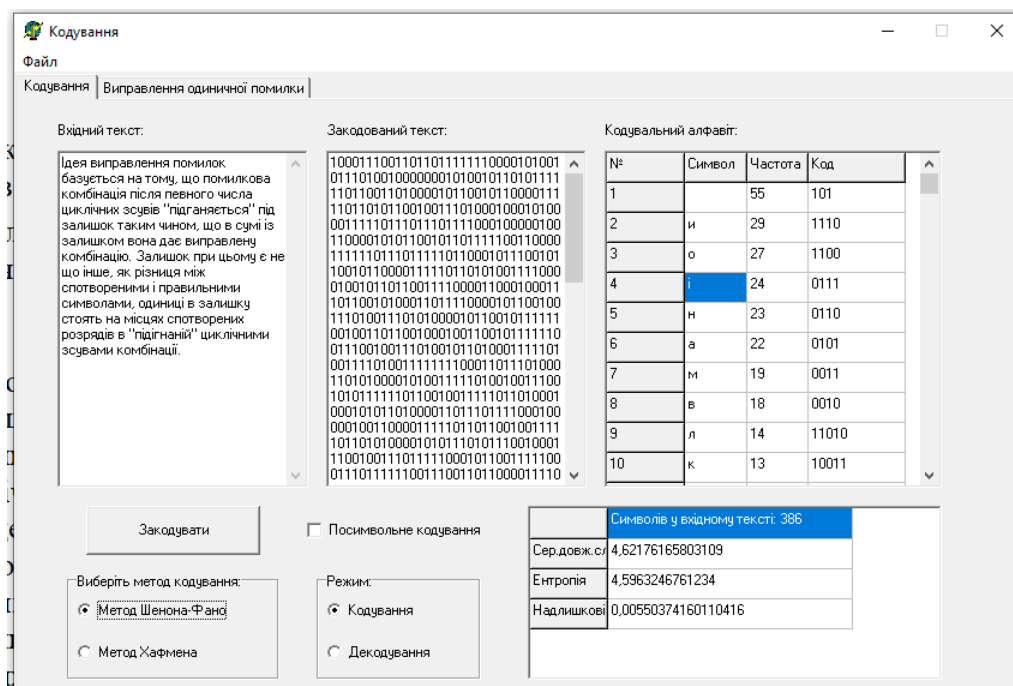


Рис.2.3. Приклад кодування по алгоритму Шенона-Фано

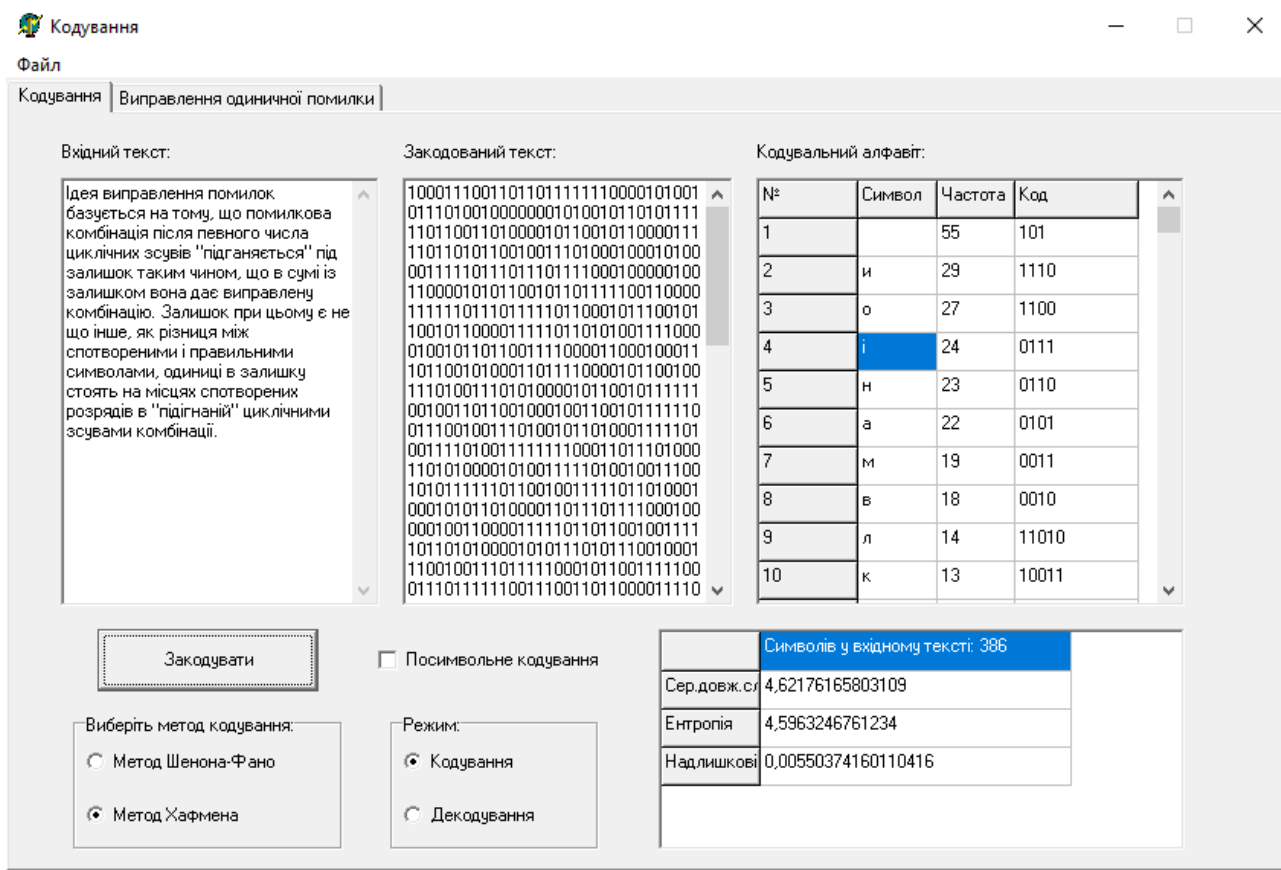


Рис.2.4. Приклад кодування по алгоритму Хафмена

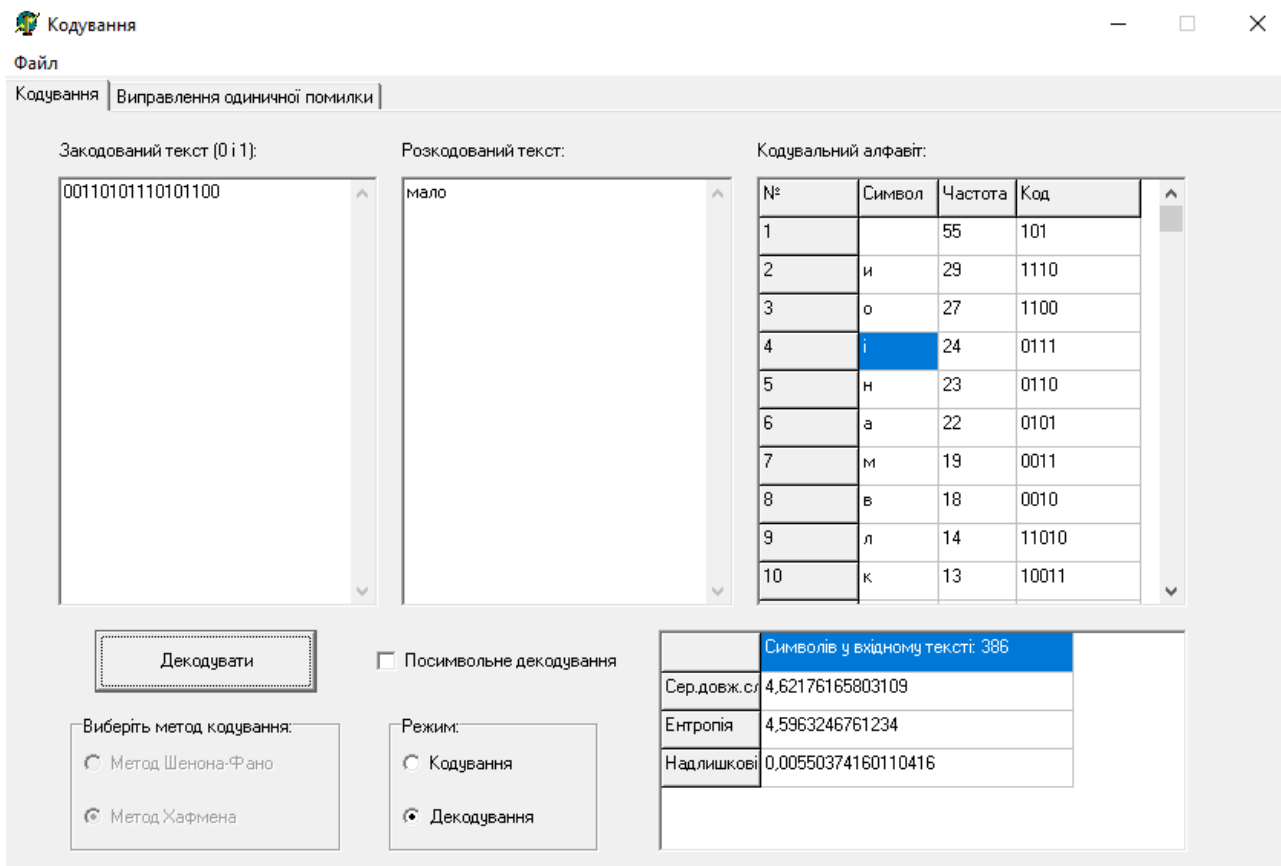


Рис.2.5. Приклад декодування

На рис.2.6.-2. Представлено можливі дії з поліноміальними кодами.

Кодування

Файл

Кодування Виправлення одиначної помилки

Кількість вхідних бітів

4

Вхідна послідовність бітів

1	0	1	0
---	---	---	---

Побудувати циклічний код

Циклічний код

1	0	1	0	0	1	1
---	---	---	---	---	---	---

Рис.2.6. Фрагмент побудови циклічного коду (7,4)

Натиснувши клавішу V, можна одержати номер помилкового біту при його наявності після передачі по комунікаційному каналі.

В

Перевірочна послідовність бітів (може містити помилку)

1	0	1	1	0	1	1
---	---	---	---	---	---	---

Пошук помилки

помилка в 4 біті

Виправлена послідовність бітів

1	0	1	0	0	1	1
---	---	---	---	---	---	---

Рис.2.7. Фрагмент моделювання і пошуку помилки в циклічному коді (7,4)

Кодування

Файл

Кодування | Виправлення одичної помилки

Кількість вхідних бітів

4

Вхідна послідовність бітів

1	2	3	4
1	0	1	0

Побудувати циклічний код

Циклічний код

1	2	3	4	5	6	7
1	0	1	0	0	1	1

V

Перевірочна послідовність бітів (може містити помилку)

1	2	3	4	5	6	7
1	0	1	1	0	1	1

Пошук помилки

помилка в 4 біті

Виправлена послідовність бітів

1	2	3	4	5	6	7
1	0	1	0	0	1	1

Рис.2.8. Загальний вид кодування і пошуку помилки в циклічному коді (7,4)

Кодування

Файл

Кодування | Виправлення одиничної помилки

Кількість вхідних бітів

4

Вхідна послідовність бітів

1	2	3	4
1	0	1	0

Побудувати циклічний код

Циклічний код

1	2	3	4	5	6	7
1	0	1	0	0	1	1

V

Перевірочна послідовність бітів (може містити помилку)

1	2	3	4	5	6	7
1	0	1	0	0	1	1

Пошук помилки

Помилка немає

Виправлена послідовність бітів

1	2	3	4	5	6	7
1	0	1	0	0	1	1

Рис.2.9. Загальний вид кодування і пошуку помилки в циклічному коді (7,4) при її відсутності

При наявності двох помилок і більше результат перевірки непередбачуваний.

ВИСНОВКИ

В даній курсовій роботі розглянуте питання економного і завадостійкого кодування, що є надзвичайно актуальним в теперішній період. За допомогою кодування є можливість значно скоротити об'єм інформації, яка передається по інформаційному каналу, забезпечивши цим кращу швидкість та якість передачі інформації. Даний розроблений програмний продукт дозволяє кодувати інформації, декодувати і також виправляти одиночну помилку в циклічних кодах Хемінга (7,4). Курсова робота дала можливість краще пізнати кодування, глибше вивчити методи та способи кодування, декодування.

ПЕРЕЛІК ПОСИЛАНЬ

1. Богдан Подлевський, Роман Рикалюк. Теорія інформації: Підручник. – Львів: ЛНУ, 2018, – 342 с.
2. Богдан Подлевський, Роман Рикалюк. Теорія інформації в задачах: Підручник. – Київ: «Центр учбової літератури», 2021, – 271 с.
3. Нікольський Ю.В., Пасічник В.В., Щербина Ю.М. Дискретна математика. – К.: Видавнича група BHV. – 2007. – 368 с.:іл.
4. В.И. Шульгин. Основы теории передачи информации. Ч. I. Экономное кодирование. – Учеб. пособие. – Харьков: Нац. аэрокосм. ун-т «Харьк. авиац. ин-т», 2003. – 102 с.
5. В.И. Шульгин. Основы теории передачи информации. Ч.2. Помехоустойчивое кодирование. – Учеб. пособие. – Харьков: Нац. аэрокосм. ун-т «Харьк. авиац. ин-т», 2003. – 87с.

ДОДАТОК А

Код програми: Файл Unit1.pas

```
unit Unit1;
interface
uses
  Windows, Messages, SysUtils, Classes, Graphics, Controls, Forms, Dialogs,
  StdCtrls, Menus, Grids, ExtCtrls, Math, ComCtrls, Spin;
type
  charcount = record //інформаційна частина:
    ch:char; //символ
    count:integer; //і скільки разів він зустрічається у тексті
    code:string[255];
  end;
type
  TForm1 = class(TForm)
    PageControl1: TPageControl;
    TabSheet1: TTabSheet;
    TabSheet2: TTabSheet;
    Label1: TLabel;
    Label2: TLabel;
    Label3: TLabel;
    Memo1: TMemo;
    Memo2: TMemo;
    Button1: TButton;
    StringGrid1: TStringGrid;
    RadioGroup1: TRadioGroup;
    StringGrid2: TStringGrid;
    CheckBox1: TCheckBox;
    RadioGroup2: TRadioGroup;
    OpenFileDialog1: TOpenDialog;
    MainMenu1: TMainMenu;
    N1: TMenuItem;
    N2: TMenuItem;
    N3: TMenuItem;
    N4: TMenuItem;
    Label4: TLabel;
    Button2: TButton;
    Label5: TLabel;
    Button3: TButton;
    StringGrid3: TStringGrid;
```

```

SpinEdit1: TSpinEdit;
Label6: TLabel;
StringGrid4: TStringGrid;
Label7: TLabel;
Label8: TLabel;
StringGrid5: TStringGrid;
StringGrid6: TStringGrid;
Label9: TLabel;
Button4: TButton;
procedure Button1Click(Sender: TObject);
procedure FormCreate(Sender: TObject);
procedure N4Click(Sender: TObject);
procedure N2Click(Sender: TObject);
procedure Memo1Change(Sender: TObject);
procedure RadioGroup1Click(Sender: TObject);
procedure CheckBox1Click(Sender: TObject);
procedure RadioGroup2Click(Sender: TObject);
procedure Button2Click(Sender: TObject);
procedure Button3Click(Sender: TObject);
procedure SpinEdit1Change(Sender: TObject);
procedure Button4Click(Sender: TObject);
private
    { Private declarations }
public
end;
var
    Form1: TForm1;
    num: array [0..255] of charcount;
    //Глобальний масив num містить код для кожного символу,
    //який зустрівся у тексті, частоту зустрічання цього
    //символа у тексті і власне сам символ. Масив має
    //n елементів, тобто n різних символів, які є у тексті.
implementation
{$R *.DFM}
//Кодування методом Шенона-Фано
procedure BuildCode(n1,n2:integer);

```

{Процедура, яка ділить задану гілку на дві півгілки, при чому кожному елементу лівої гілки до коду додається 0, а кожному елементу правої гілки додається одиниця. На вхід подаються 2 параметри: n1 показує початок гілки, n2 - кінець гілки.

Алгоритм такий: спочатку сортуємо всі елементи всередині гілки за спаданням. Потім беремо перший елемент. Якщо кількість його входжень більша, ніж половина від загальної кількості символів, то цей елемент потрапляє до лівої гілки, усі інші - до правої. Якщо ж ні, то беремо перші 2 елементи, потім перші 3 і т.д., доки

не перевищимо половину від заг. кількості символів. Далі рекурсія - викликаємо цю саму процедуру по черзі для правої і лівої півгілки. Алгоритм закінчується, якщо у обох гілках залишиться по одному елементу.

```
}  
var i,j,cursum:integer;  
halfsum:real;  
k:charcount;  
s1,s2:string;  
begin  
if n1=n2 then exit; //виходимо, якщо у гілці лише 1 символ  
for i:=n1 to n2-1 do //сортуємо гілку за спаданням  
  for j:=i to n2 do  
    if num[i].count<num[j].count then begin  
      k:=num[i];  
      num[i]:=num[j];  
      num[j]:=k;  
    end;  
halfsum:=0;  
for i:=n1 to n2 do  
  halfsum:=halfsum+num[i].count;  
  
halfsum:=halfsum/2; //половина кількості символів у гілці  
  
i:=n1; cursum:=0; //беремо спочатку один елемент  
repeat //якщо його кількість входжень менша за половину  
  //всієї кількості символів у гілці,  
  cursum:=cursum+num[i].count; //то беремо ще один  
  inc (i);  
until cursum>halfsum; //якщо сумарна кількість вибраних  
//символів більша за вказане число, то виходимо з циклу.  
//Вибрані символи утворили ліву півгілку, а всі решта - праву.  
  
s1:=""; s2:="";  
//далі дописуємо 0 до коду кожного символу лівої півгілки  
//і 1 для правої.  
if n2-n1=1 then begin //якщо у гілці лише 2 символи  
  s1:=num[n1].ch; s2:=num[n2].ch;  
  num[n1].code:=num[n1].code+'0'; //то розподіляємо по 1 символу  
  num[n2].code:=num[n2].code+'1'; //у ліву і праву півгілки  
  i:=n2; end  
else //якщо у гілці більше, ніж 2 символи  
for j:=n1 to n2 do //тоді відраховуємо половину  
  if j<i then begin num[j].code:=num[j].code+'0'; s1:=s1+num[j].ch; end
```

```

else begin num[j].code:=num[j].code+'1'; s2:=s2+num[j].ch; end;

//тепер повторюємо цю саму процедуру по черзі
//для лівої та правої півгілки
BuildCode(n1,i-1);
BuildCode(i,n2);

end;

//Кодування методом Хафмена
{Алгоритм такий: процедура на вході отримує кодову стрічку, яка має вигляд:
s1#&${n1;%~s2#&${n2;%~...;%~sk#&${nk;%~
де s1..sk - символ або група символів, n1..nk – сумарна частота відповідної групи символів. Процедура
записує ці дані у масив, сортує його за зростанням відносно частоти кожної групи символів, а потім об'єднує
два символи з найменшими значеннями частоти у єдину групу, а їх частоти додаються. Виходить нова кодова
стрічка, для якої знову виконуються усі вищевказані дії. Процес триває доти, доки не об'єднуються в єдину
групу усі символи, які були у початковій кодовій стрічці.
}
procedure Hafmen(st:string; var codes: array of string);
type hafmentree=record
  c: string;
  sum:integer;
end;
var mas: array [0..255] of hafmentree;
i,j,n,k: integer; x:string;
begin
  n:=0; i:=1;
  //Спершу розбиваємо вхідну стрічку на пари вигляду
  //s1#&${n1 і т.д. Застосовуємо алгоритм виділення слів.
  while i<=length(st) do begin
    if i>length(st) then break;
    j:=i+1;
    while (j<=length(st)) and not(copy(st,j,3)=';%~') do j:=j+1;
    n:=n+1;
    mas[n-1].c:=copy(st,i,j-i);
    i:=j+3;
  end;
  //Потім кожну таку пару розбиваємо на 2 частини: c - послідовність
  //символів, а sum - сумарна частота усіх символів у даній
  //послідовності. Розділювачем служить набір з 4 символів #&${^
  for i:=0 to n-1 do
    for j:=1 to length(mas[i].c)-3 do

```

```

if copy(mas[i].c,j,4)='#&$^' then begin
    mas[i].sum:=strtoint(copy(mas[i].c,j+4,length(mas[i].c)-j-3));
    mas[i].c:=copy(mas[i].c,1,j-1);
    break;
end;

if n=1 then exit; //виходимо, якщо в масиві лише один елемент, бо
//в такому разі більше нема елементів, які треба об'єднувати.

//Сортуємо масив mas за зростанням
for i:=0 to n-2 do
    for j:=i to n-1 do
        if mas[i].sum>mas[j].sum then begin
            k:=mas[i].sum; mas[i].sum:=mas[j].sum; mas[j].sum:=k;
            x:=mas[i].c; mas[i].c:=mas[j].c; mas[j].c:=x;
        end;

//Усім символам в першому елементі присвоюємо код 0,
//а в другому - код 1.
for i:=1 to length(mas[0].c) do
    codes[ord(mas[0].c[i])]:=codes[ord(mas[0].c[i])]+'0';
for i:=1 to length(mas[1].c) do
    codes[ord(mas[1].c[i])]:=codes[ord(mas[1].c[i])]+'1';
//Об'єднуємо два елементи з найменшими частотами в один
//(елементи з номерами 0 та 1).
mas[0].c:=mas[0].c+mas[1].c;
mas[0].sum:=mas[0].sum+mas[1].sum;
//Видаляємо другий елемент, залишаючи тільки перший.
for i:=1 to n-2 do begin
    mas[i].c:=mas[i+1].c;
    mas[i].sum:=mas[i+1].sum;
end;
n:=n-1; //зменшуємо кількість елементів масиву на 1

//З нового масиву утворюємо нову кодову стрічку
x:="";
for i:=0 to n-1 do
    x:=x+mas[i].c+'#&$^'+inttostr(mas[i].sum)+';';

//Повторюємо цю саму процедуру для кодової стрічки доти,
//доки усі її елементи не об'єднуються в один.
Hafmen(x,codes);

```

```

end;

//ОСНОВНА ПРОЦЕДУРА КОДУВАННЯ
procedure EncodeText();
var
i,j,n,allsymbols:integer; s,st:string;
symb:array [0..255] of integer;
codes: array [0..255] of string;
k:charcount;
abetka: set of char;
ser,entrop,nadl:real;
begin
if form1.memo1.text="" then begin //якщо текст для кодування не введено
    MessageDlg('Спочатку введіть текст для кодування!',mtWarning,[mbOk],0);
    exit;//то виходимо з процедури
end;
//у масиві symb буде вказано, скільки разів у тексті
//зустрічається кожен символ
for i:=0 to 255 do begin symb[i]:=0; num[i].count:=0; num[i].code:=""; codes[i]:=""; end;
//зануляємо масиви symb і num
form1.memo2.lines.clear;
for i:=1 to length(form1.memo1.text) do begin
    if not(form1.memo1.text[i] in abetka) then include(abetka,form1.memo1.text[i]);
    //якщо даний символ зустрівся уперше, то заносимо його до
    //множини вхідного алфавіту
    symb[ord(form1.memo1.text[i])]:=symb[ord(form1.memo1.text[i])+1];
    //збільшуємо лічильник даного символу на 1.
end;
n:=0;
for i:=0 to 255 do
    if symb[i]<>0 then begin
        n:=n+1;
        num[n-1].ch:=chr(i);
        num[n-1].count:=symb[i];
    end;
//тепер у масиві num вказано, скільки разів зустрічається
//кожен символ.
//n - кількість різних символів у тексті.

for i:=0 to n-2 do //сортуємо масив num за спаданням
    for j:=i to n-1 do
        if num[i].count<num[j].count then begin

```

```

        k:=num[i];
        num[i]:=num[j];
        num[j]:=k;
    end;
allsymbols:=0;
for i:=0 to n-1 do //рачуємо, скільки всього символів
    allsymbols:=allsymbols+num[i].count; //у нашому тексті
form1.stringgrid2.Cells[1,0]:='Символів у вхідному тексті: '+inttostr(allsymbols);

if n=1 then num[0].code:='0';//якщо у стрічці лише 1 символ,то його код 0

//Якщо метод Шенона-Фано, то будуємо код для кожного символу
if form1.radiogroup1.ItemIndex=0 then BuildCode(0,n-1)
else begin //Якщо ж метод Хафмена,
    st:=''; //то будуємо кодувальну стрічку
    for i:=0 to n-1 do
        st:=st+num[i].ch+'#&$^'+inttostr(num[i].count)+';%~';
    Hafmen(st,codes); //код кожного символу - у масиві codes
    //Тепер код кожного символу треба записати у зворотньому порядку,
    //бо код рахується від основи дерева, а не від листків.
    for i:=0 to 255 do begin
        st:='';
        for j:=length(codes[i]) downto 1 do
            st:=st+codes[i][j];
        codes[i]:=st;
    end;
end;

//Якщо це був метод Хафмена, то треба усі значення кодів з масиву
//codes скопіювати у масив num, щоб вивести їх у таблицю.
//Якщо у тексті лише 1 символ, то він уже раніше отримав
//код 0 і нема змісту знову псувати його код.
if (form1.radiogroup1.ItemIndex=1) and (n>1) then begin
    for i:=0 to n-1 do
        num[i].code:=codes[ord(num[i].ch)];
end;

form1.stringgrid1.RowCount:=n+1;
for i:=0 to n-1 do begin//Виводимо код кожного символу
    //у таблиці
    form1.stringgrid1.cells[0,i+1]:=inttostr(i+1);//порядковий номер
    form1.stringgrid1.cells[1,i+1]:=num[i].ch;//сам символ
end;

```

```

form1.stringgrid1.cells[2,i+1]:=inttostr(num[i].count);//його частота
form1.stringgrid1.cells[3,i+1]:=num[i].code;//його код
end;

//Рахуємо середню довжину кодового слова
ser:=0;
for i:=0 to n-1 do
    ser:=ser+(num[i].count/allsymbols)*length(num[i].code);
form1.stringgrid2.RowCount:=2;
form1.stringgrid2.cells[0,1]:='Сер.довж.слова';
form1.stringgrid2.cells[1,1]:=floattostr(ser);
//Рахуємо ентропію джерела
entrop:=0;
for i:=0 to n-1 do
    entrop:=entrop-(num[i].count/allsymbols)*Log2(num[i].count/allsymbols);
form1.stringgrid2.RowCount:=3;
form1.stringgrid2.cells[0,2]:='Ентропія';
form1.stringgrid2.cells[1,2]:=floattostr(entrop);
//Рахуємо надлишковість коду
nadl:=1-(entrop/ser);
form1.stringgrid2.RowCount:=4;
form1.stringgrid2.cells[0,3]:='Надлишковість коду';
form1.stringgrid2.cells[1,3]:=floattostr(nadl);

//Копіюємо код кожного символу до масиву codes, де кожному
//номеру символу з таблиці ASCII відповідає двійковий код
//у символній формі.
//Якщо ж символ у тексті не зустрічається, то кодувати його
//нема змісту і відповідно його код - порожній рядок.
//Цей масив кращий, ніж масив num, бо тут швидше можна знайти
//код потрібного символу - за його ASCII-кодом. Таким чином,
//не потрібно щоразу проходити весь масив у пошуках елемента.
for i:=0 to n-1 do
    codes[ord(num[i].ch)]:=num[i].code;

st:="";//стрічка, куди будемо записувати закодоване повідомлення
for i:=1 to length(form1.memo1.text) do
    st:=st+codes[ord(form1.memo1.text[i])];

form1.memo2.text:="";
form1.memo2.text:=st; //виводимо закодоване повідомлення
form1.RadioGroup2.enabled:=true; //кнопка "Декодер" стає доступною

```


end;

//ОСНОВНА ПРОЦЕДУРА ДЕКОДУВАННЯ

{Алгоритм: проходимо посимвольно введений текст для декодування, який складається лише з нулів та одиниць. Спочатку j=1. Починаючи від j-го символу беремо 1,2,3 або більше символів і перевіряємо, чи складають вони якесь кодове слово. Якщо так, то дане слово замінюємо на символ, який ним закодований. j ставимо на символ, наступний після кінця даного кодового слова, і рухаємося далі, шукаючи наступне кодове слово. Якщо досягнули кінця стрічки і не утворилося жодне кодове слово, то друкуємо тире - це означає, що даний текст неможливо розкодувати.

```
}  
procedure DecodeText();  
var i,j,m,n:integer;  
wordfound:boolean;//змінна,яка вказує,чи знайшли ми кодове слово  
begin  
if form1.memo1.text="" then begin //якщо текст для декодування не введено  
  MessageDlg('Спочатку введіть текст для декодування!',mtWarning,[mbOk],0);  
  exit; //то виходимо з процедури  
end;  
//Як знати, скільки елементів у масиві num? Тобто скільки  
//є різних кодових слів. Їх є стільки, скільки рядків є  
//у таблиці кодувального алфавіту (крім першого допоміжного  
//рядка).  
n:=form1.StringGrid1.RowCount-1;  
form1.memo2.text:="";  
  
j:=1; //j вказує на перший символ даного слова  
for i:=1 to length(form1.memo1.text) do begin  
  //Якщо якийсь із символів тексту для декодування  
  //не є нулем чи одиницею  
  if (form1.memo1.text[i]<>'0') and (form1.memo1.text[i]<>'1') then begin  
    MessageDlg('Текст для декодування може містити лише символи 0 і 1!',mtWarning,[mbOk],0);  
    exit; //то виходимо з процедури  
  end;  
  wordfound:=false;//ознака того, що кодове слово розпізнано  
  for m:=0 to n do  
    if copy(form1.memo1.text,j,i-j+1)=num[m].code then begin  
      wordfound:=true;//кодове слово розпізнано  
      form1.memo2.text:=form1.memo2.text+num[m].ch;  
      j:=i+1;  
      break;  
    end;  
end;
```

```

end;

//якщо якийсь кодове слово не знайшли, то на його місці
//ставимо тире
if not wordfound then form1.memo2.text:=form1.memo2.text+'-';

end;

//Кнопка "Закодувати"
procedure TForm1.Button1Click(Sender: TObject);
begin
if radiogroup2.ItemIndex=0 then EncodeText()
  else DecodeText();
end;

procedure TForm1.FormCreate(Sender: TObject);
begin
stringgrid1.Cells[0,0]:='№';
stringgrid1.Cells[1,0]:='Символ';
stringgrid1.Cells[2,0]:='Частота';
stringgrid1.Cells[3,0]:='Код';
stringgrid3.Cells[0,0]:='1';
end;

//Меню "Вихід"
procedure TForm1.N4Click(Sender: TObject);
begin
form1.Close;
end;

//Меню "Відкрити", яке дозволяє відкрити текстовий файл для кодування
procedure TForm1.N2Click(Sender: TObject);
begin
opendialog1.Execute; //вікно вибору файла
if opendialog1.FileName="" then exit;
memo1.lines.clear();
memo1.lines.LoadFromFile(opendialog1.FileName);
end;

procedure TForm1.Memo1Change(Sender: TObject);
begin
//Якщо увімкнене посимвольне кодування, то при натисканні

```

```

//клавіші на текстовому полі щоразу відбувається
//повторне кодування чи декодування.
if checkbox1.Checked then
    if radiogroup2.ItemIndex=0 then EncodeText()
    else DecodeText();
end;

procedure TForm1.RadioGroup1Click(Sender: TObject);
begin
    //Якщо увімкнене посимвольне кодування, то при зміні
    //зміні методу кодування вхідний текст автоматично заново
    //кодується новим методом.
    if checkbox1.Checked then EncodeText();
end;

procedure TForm1.CheckBox1Click(Sender: TObject);
begin
    if checkbox1.Checked then
        if radiogroup2.ItemIndex=0 then EncodeText()
        else DecodeText();
    end;

    //Обираємо режим: кодування чи декодування
    procedure TForm1.RadioGroup2Click(Sender: TObject);
    begin
        if radiogroup2.ItemIndex=0 then begin
            button1.Caption:='Закодувати';
            checkbox1.Caption:='Посимвольне кодування';
            label1.Caption:='Вхідний текст:';
            label2.Caption:='Закодований текст:';
            radiogroup1.Enabled:=true;
            memo1.lines.clear;
            memo2.lines.clear;
        end else begin
            button1.Caption:='Декодувати';
            checkbox1.Caption:='Посимвольне декодування';
            label1.Caption:='Закодований текст (0 і 1):';
            label2.Caption:='Розкодований текст:';
            radiogroup1.Enabled:=false;
            memo1.lines.clear;
            memo2.lines.clear;
        end;
    end;
end;

```

```

end;

//процедура зсуву вліво на 1 біт
procedure left_bits(var bits:string);
var ch:char;
    i:integer;
begin
ch:=bits[1];
for i:=1 to length(bits)-1 do
    bits[i]:=bits[i+1];
bits[length(bits)]:=ch;
end;

//процедура зсуву вправо на 1 біт
procedure right_bits(var bits:string);
var ch:char;
    i:integer;
begin
ch:=bits[length(bits)];
for i:=length(bits) downto 2 do
    bits[i]:=bits[i-1];
bits[1]:=ch;
end;

//функція обчислення ваги
function weight(bits:string):integer;
var i:integer;
begin
result:=0;
for i:=1 to length(bits) do
    result:=result+strtoint(bits[i]);
end;

//функція додавання за модулем 2
function add_mod_2(bits,bits_ost:string):string;
var i,j:integer;
begin
result:="";
for i:=1 to length(bits) do
    result:=result+' ';
j:=length(bits);
i:=length(bits_ost);

```

```

while i>0 do
    begin
        if bits[j]<>bits_ost[i] then
            result[j]:='1'
        else
            result[j]:='0';
        i:=i-1;
        j:=j-1;
    end;
while j>0 do
    begin
        result[j]:=bits[j];
        j:=j-1;
    end;
end;

//функція знаходження остачі
function ostacha(bits1,bits2:string):string;
var i,j:integer;
    tmp:string;
begin
    j:=1;
    tmp:="";
    repeat
        i:=1;
        while (i<=length(tmp))and((tmp[i]='0')) do
            delete(tmp,i,1);
        while (length(tmp)<length(bits2))and(j<=length(bits1)) do
            begin
                tmp:=tmp+bits1[j];
                j:=j+1;
            end;
        if length(tmp)=length(bits2) then
            tmp:=add_mod_2(tmp,bits2)
        until j>length(bits1);
        result:=tmp;
    end;

//функція отримання незвідного многочлена
function get_nezvid_mnog(n:integer):string;
begin
    if n=2 then

```

```

        result:='111'
    else if n=3 then
        result:='1011'
    else if n=4 then
        result:='11001'
    else if n=5 then
        result:='100101'
    else if n=6 then
        result:='1000011'
    else if n=7 then
        result:='10001001'
    else if n=8 then
        result:='100011101'
    else if n=9 then
        result:='1000010001';
    end;

```

```

//функція отримання циклічного коду
function make_code(input_bits:string):string;
var ni,nk,i:integer;
    mn,ost,tmp:string;
begin
    ni:=length(input_bits);
    nk:=round(ln(ni+1)/ln(2));
    nk:=round(ln(ni+1+nk)/ln(2));
    tmp:=input_bits;
    for i:=1 to nk do
        tmp:=tmp+'0';
    mn:=get_nezvid_mnog(nk);
    ost:=ostacha(tmp,mn);
    delete(ost,1,length(ost)-nk);
    result:=input_bits;
    for i:=1 to nk-length(ost) do
        result:=result+'0';
    result:=result+ost;
end;

```

```

//функція виявлення одиничної помилки
function find_error(var input_bits:string):integer;
var n,nk,w,n_s,i:integer;
    mn,ost:string;
begin

```

```

n:=length(input_bits);
nk:=round(ln(n+1)/ln(2));
mn:=get_nezvid_mnog(nk);
ost:=ostacha(input_bits,mn);
w:=weight(ost);
if w=0 then
    result:=-1
else
    begin
        n_s:=0;
        while w>1 do
            begin
                n_s:=n_s+1;
                left_bits(input_bits);
                ost:=ostacha(input_bits,mn);
                w:=weight(ost);
            end;
            i:=1;
            while (ost[i]='0')and (i<=length(ost)) do
                delete(ost,i,1);
            input_bits:=add_mod_2(input_bits,ost);
            for i:=1 to n_s do
                right_bits(input_bits);

            if n_s=0 then
                result:=n-length(ost)+1
            else
                result:=n_s;
            end;
        end;
    end;
end;

```

```

procedure TForm1.Button2Click(Sender: TObject);
var input_bits,code:string;
    n,i:integer;
begin
    n:=SpinEdit1.value;
    input_bits:="";
    for i:=1 to n do
        input_bits:=input_bits+StringGrid3.cells[i-1,1];
    code:=make_code(input_bits);
    if code<>" then

```

```

begin
StringGrid4.ColCount:=length(code);
for i:=1 to length(code) do
begin
Stringgrid4.Cells[i-1,1]:=code[i];
Stringgrid4.Cells[i-1,0]:=inttostr(i);
end;
end
else ShowMessage('Неможливо побудувати циклічний код')
end;

procedure TForm1.Button3Click(Sender: TObject);
var n_e,i:integer;
i_b:string;
begin
i_b:="";
for i:=1 to Stringgrid5.colcount do
i_b:=i_b+StringGrid5.Cells[i-1,1];
n_e:=find_error(i_b);
if n_e=-1 then
label5.caption:='Помилка немає'
else
label5.caption:='помилка в '+inttostr(n_e)+' бітi';
StringGrid6.ColCount:=length(i_b);
for i:=1 to length(i_b) do
begin
StringGrid6.Cells[i-1,0]:=inttostr(i);
StringGrid6.Cells[i-1,1]:=i_b[i];
end;
end;

procedure TForm1.SpinEdit1Change(Sender: TObject);
var i:integer;
begin
if SpinEdit1.Value<1 then
Spinedit1.Value:=1;
Stringgrid3.ColCount:=SpinEdit1.Value;
for i:=1 to SpinEdit1.Value do
StringGrid3.Cells[i-1,0]:=Inttostr(i);
end;

procedure TForm1.Button4Click(Sender: TObject);

```



```
var i:integer;
begin
Stringgrid5.ColCount:=StringGrid4.ColCount;
for i:=1 to StringGrid4.ColCount do
  begin
    Stringgrid5.Cells[i-1,1]:=Stringgrid4.Cells[i-1,1];
    Stringgrid5.Cells[i-1,0]:=inttostr(i);
  end;
end;

end.
```